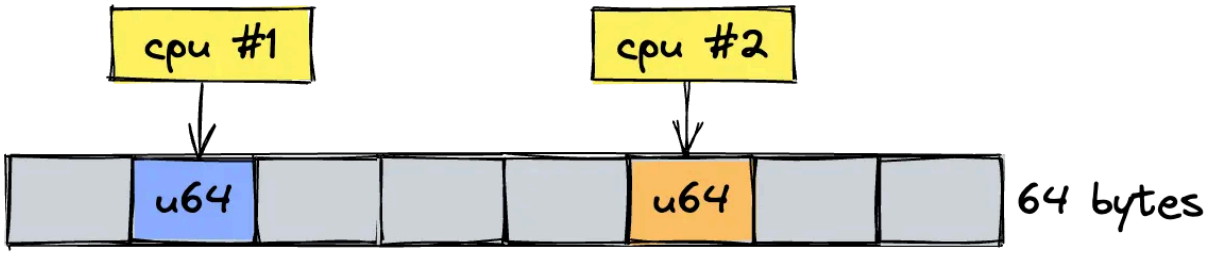
ALGOMESH

WHY WE BUILT IT : Normal Trading Introduces too much time lag , The technology HFT's use is still not open to public and open-source projects available are hard to use , so we decided to create an open-source trading engine for crypto that is human friendly

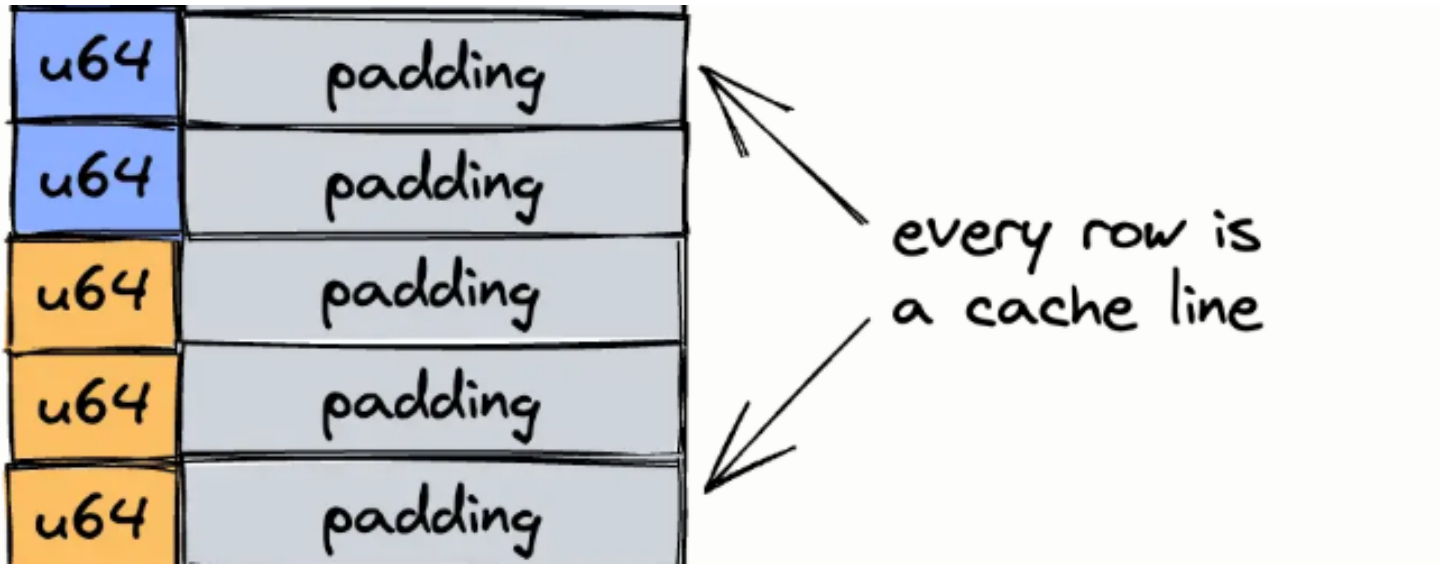
WHAT HAPPENS INSIDE (SIMPLIFIED) :
DATA COMES → INDICATORS GET UPDATED → EVALUATE STRATEGY → PUBLISH RESULT

How we optimised it to extract as much performance as we can ?

PROBLEM : **False Sharing & Cache-Line Alignment:** Modern CPU caches (L1/L2/L3) transfer data in 64-byte cache lines. When two threads running on separate CPU cores modify independent variables residing on the same 64-byte cache line, the CPU cache coherence protocol (e.g., MESI) repeatedly invalidates the cache line across cores, causing severe performance degradation known as **false sharing**.
we eliminate this using alignas(64) in C++20 to force structs or variables onto distinct 64-byte memory boundaries.

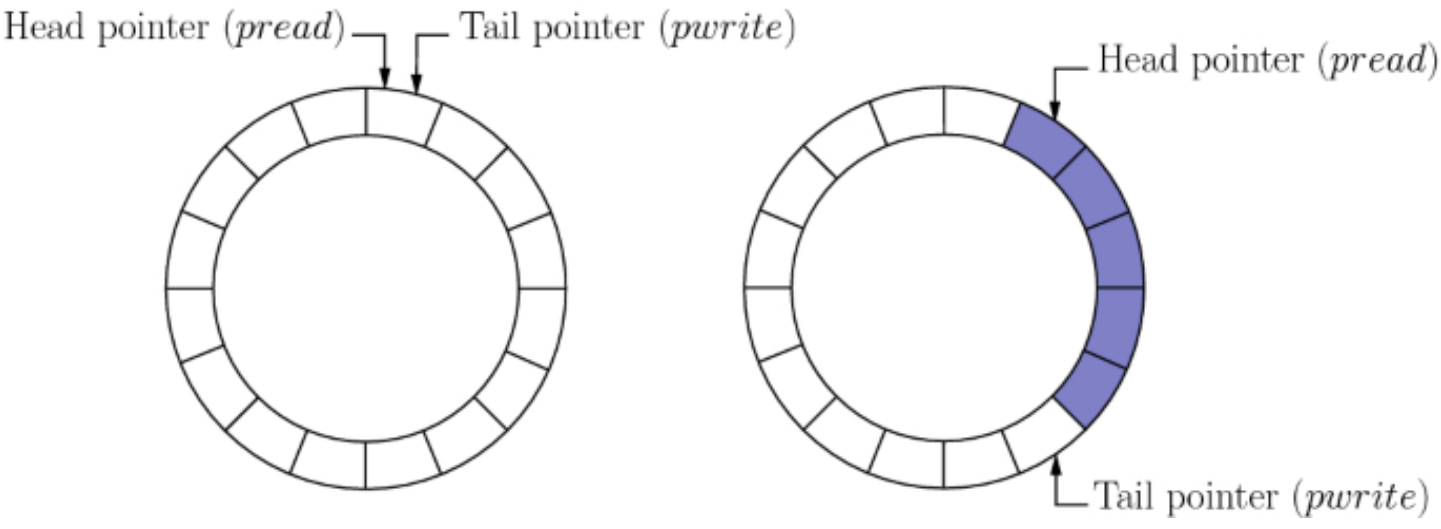


SOLUTION :



PROBLEM : Branch Prediction: Modern CPUs use speculative execution to predict branch outcomes (if/else). A misprediction flushes the execution pipeline (~15–20 cycles penalty). High branch prediction accuracy (98.92%) is achieved by writing branchless code (e.g., bitwise operations, lookup tables, [[likely]] / [[unlikely]] attributes in C++20)

Lock-Free SPSC Ring Buffer: A Single-Producer Single-Consumer queue that allows concurrent read/write operations without mutexes or lock contention. It relies on atomic head/tail indices using atomic memory orders: std::memory_order_release when publishing items and std::memory_order_acquire when consuming items.



Slab Allocator: Custom memory allocator designed to eliminate internal/external fragmentation and O(N) system call overhead (malloc/free). It pre-allocates a contiguous memory pool divided into fixed-size chunks ("slabs"), enabling O(1) allocation and deallocation via a free list.

